

Sprint 1

Marchini Claudio, Tomasi Cesare

Introduction

This sprint builds on top of the work of sprint 0 and aims to build the core business of the system, in particular:

- The main *cargoservice* actor
- The *sonar* subsystem
- The *IOPort* and *pushbutton* interfaces
- Concrete implementations of the following interfaces: *ISlot*, *IHold*, *IPosition*

Requirement Analysis

The requirement analysis has already been specified in the sprint 0, this particular sprint does not require anything else to be specified in this section.

Problem Analysis

The **cargoservice** actor works as the main component of the system, the requirement analysis specifies the workflow of the component, here we will specify it again in order to focus on many details with more precision:

1. It listens for a **request to load** received by the *pushbutton*
2. It checks the state of the **IOPort**, and decides which answer to send back, as defined in the sprint 0



3. If a slot is free, it waits for the container to be placed in the *sensor area* and then delegates the *cargorobot* service to move the container to **slot-5**
4. It then moves the container from **slot-5** to the reserved slot again with the use of the **cargorobot**

```
QActor cargospace context ctxcargospace {
  [# val TimeoutMillis = 30000 #]
  [# val ReservedSlot = 0 #]

  State s0 disengaged {
    // Wait for a request to load

    if [# ioport occupied #] {
      //send retrylater
    } else if [# slots occupied #] {
      //send rejected
    } else {
      //reserve slot
      //set reserved slot var
      //send accepted(slot id)
      Goto engaged
    }
  }

  State engaged {
    //call blink led service
  }

  Transition t0
    whenTime TimeoutMillis -> disengaged
    whenMessage IOPortDeposited -> moveRobot

  State moveRobot {
    // call robot service from IOPort to slot 5
    // wait 3s to mark the container
    // call robot service from slot 5 to slot ReservedSlot

    //stop blink led service
    Goto disengaged
  }
}
```



The **sonar** is a HC-SR04 and is connected to a *Raspberry Pi Pico W*, by nature the hardware has a few characteristics:

- The raspberry is too lightweight to support a full JVM needed to run *qak* directly, so we have to use either cpp or micropython as the language
- Since *qak* is not a possibility the communication must be implemented with a protocol, preferably one that *qak* supports out of the box
- *qak* supports a plethora of protocols, TCP, CoAP, MQTT, ecc..

The software house has decided to use **MQTT** as the protocol of choice because:

- It uses a centralized broker/client system, allowing the system to easily change or add devices in the future (multiple IOPorts or a device substitution), otherwise at least one of the nodes should know how to reach the other one, which is not always granted
- The broker is not particularly resource intensive and can easily be deployed along the rest of the service, likely in the same node as the main *cargoservice* actor
- The topic system keeps messages from various nodes neatly separated



```

def connectWifi():
    #connect to wifi

    print("Connected:", wlan.ifconfig())

def connectMqtt():
    # create Mqtt client
    # connect to Mqtt broker
    return client

def measureDistance():
    # measure distance using sonar
    return distance

TRIG = Pin(3, Pin.OUT)
ECHO = Pin(2, Pin.IN)

connectWifi()
client = connectMqtt()

while True:
    try:
        d = measureDistance()
        if d is not None:
            # create formatted msg
            client.publish(TOPIC, msg.encode() )

            time.sleep(1)
    except Exception as e:
        print("Exception: ", type(e).__name__, e)

```

- We have assumed that the Board will be connected through Wifi
- The configurations parameters (Wifi SSID, Password, MQTT broker, topic, ect..) will be set in a `.env` file in order to enhance reusability.
- The `msg` will be formatted as specified by the documentation of [unibo.basicomm23-1.0](#), a library developed by our software house, already used by *qak*



Once we have the raw distance data we have two options:

- Send to the rest of the system the distance, it will then be a responsibility of each

component to interpret the data accordingly as defined in the requirements.

- Create an intermediary that transforms the raw data into messages that respect the semantics of the requirements that will be sent to the rest of the system.

We decided to create an intermediary wrapper in order to have a single point where the distance is parsed, this allows to avoid repeating code, making it more robust and easily adjustable in the future.

```
Event serviceWorking : serviceWorking(X)
Event outOfService : outOfService(X)

Dispatch containerInIoPort : containerInIoPort(X)

QActor sonarwrapper context ctxcargoservice {
  State s0 initial {
    // listen to the sonar messages and send the right events/dispatch
  }
}
```

We choose to make the display messages events to permit a future expansion through extensibility. The containerInIoPort, instead is a dispatch because the only receiver should be the *cargoservice* actor.

Since the **IoPort** has to maintain the current status of the system at any moment the web page will need a **WebSocket** connection to ensure it receives updates, also receive the state of the *hold* our implementation in the requirement analysis will also need a listener that will forward any change via WebSocket to all connected clients.



```
const socket = new WebSocket("ws://localhost:8080");

const requestSpan = document.getElementById("request-result");
const statusSpan = document.getElementById("current-status");

socket.addEventListener("open", (event) => {
  //Request initial state
});

socket.addEventListener("message", (event) => {
  // Dispatch by message type and update right span
});
```

Implementations of the components from the model (the interfaces were defined in the Sprint0) were also produced:



```
public class Hold implements IHold {
    private IPosition ioPosition;
    private IPosition homePosition;

    private List<Pair<IPosition, ISlot>> slotList =
        new ArrayList<Pair<IPosition, ISlot>>();

    public Hold() {
        //lettura parametri da file esterno .properties per maggiore modularità
        int width = Config.getInt("width");
        int length = Config.getInt("length");
        int delta = Config.getInt("delta");

        ioPosition = new Position(0, width - 1);
        homePosition = new Position(0, 0);

        slotList.add(new Pair<IPosition, ISlot>(
            new Position(delta, delta), new Slot(1)));
        slotList.add(new Pair<IPosition, ISlot>(
            new Position(width - delta - 1, delta), new Slot(2)));
        slotList.add(new Pair<IPosition, ISlot>(
            new Position(delta, length - delta - 1), new Slot(3)));
        slotList.add(new Pair<IPosition, ISlot>(
            new Position(width - delta - 1, delta), new Slot(4)));
    }

    @Override
    public IPosition getIOPortPosition() {
        return ioPosition;
    }

    @Override
    public IPosition getHomePosition() {
        return homePosition;
    }

    @Override
    public List<Pair<IPosition, ISlot>> getSlots() {
        return slotList;
    }
}

public class Slot implements ISlot {
    private int id;
    private boolean occupied;
}
```



```
public Slot(int id) {
    this.id = id;
    occupied = false;
}

@Override
public int getID() {
    return this.id;
}

@Override
public boolean isOccupied() {
    return this.occupied;
}

@Override
public void setOccupied(boolean value) {
    this.occupied = value;
}
}

public class Position implements IPosition {
    private int x;
    private int y;

    public Position(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public int getX() {
        return this.x;
    }

    @Override
    public int getY() {
        return this.y;
    }
}
```

Most of the relevant parameters are read from the external configuration file **model.properties** via the utility class Config.



Test Plans

The tests for this sprint focus on the interaction of the various components that have been built. At this point we can test the flow an interaction with the user:

- In the initial state, since no hold has been occupied, when the user makes a *request to load*, the system should reply with request accepted and the status of the page should reflect that. Also once the sonar detects a container the system should transition to the *engaged* state and start to blink the *led* on the Raspberry.
- We can also test the edge cases that do not set the service to *engaged*, in particular, if the *ioport* is occupied or all slots are occupied, the display should receive a *retry later* and *rejected* message respectively, also various log messages have been added to the system to trace the behaviour.

Since these test do not simply run on a single component, unit tests are not feasible so end to end tests are required, the user can manually interact with the app and see the expected behaviour in the web gui and the physical led.

Several unit tests regarding the POJOs from the model were produced. They can be run via gradle via the following command:

```
./gradlew test
```

The project's *build.gradle* file had to be modified to add this functionality.

Deployment

The system will be deployed on docker, to keep the components separated different images will be built:



- cargoservice

- *ioport*

This allows the possibility to deploy the various components on different nodes, giving flexibility on the client. The *cargoservice* and *ioport* will be essentially the same: they will take a *tar* file of the application and run it on a *jvm* image, exposing the required ports respectively.

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /cargoservice
ADD ./build/distributions/cargoservice-1.0.tar /cargoservice
WORKDIR /cargoservice/cargoservice-1.0

EXPOSE 5000
CMD [ "./bin/cargoservice" ]
```

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /ioport
ADD ./build/distributions/ioport-1.0.tar /ioport
WORKDIR /ioport/ioport-1.0

EXPOSE 8080
CMD [ "./bin/ioport" ]
```

Lastly, to connect the two images in one cohesive system a compose file will be made, this file connects the two services internally via tcp on ports 5000/5001 on a private network, and exposes to the host only port 8080 which is needed for the web interface of the *ioport*.



```
services:
  cargoservice:
    build:
      dockerfile: ./docker/Dockerfile.cargoservice
    image: cargoservice:latest
    container_name: cargoservice

  ioport:
    build:
      dockerfile: ./docker/Dockerfile.ioport
    image: ioport:latest
    container_name: ioport
    ports:
      - 8080:8080
    depends_on:
      - cargoservice
```

If the client wishes to build and run the application, only two steps are required

1. Build the tar files by running `gradle distTar` and `gradle ioportDistTar`
2. Build and deploy the docker images by running `docker compose up --build`

Maintenance



Marchini Claudio

claudio.marchini@studio.unibo.it



Tomasi Cesare

cesare.tomasi@studio.unibo.it



SignedSnow0/CargoService

